Artificial Intelligence

Professor: Dr. Sajedi

November 2024

HW1
Implementing Tic Tac Toe Using Q-Learning

Anahita Hedayatifard

Problem 1) The number of states depends on the size of the game table. For example, a 5×5 table has 3^{25} possible states, while a 3×3 table has 3^9 states.

Increasing the size of the game table significantly raises the number of states and alters the number of winning states. This forces the algorithm to handle a larger Q-table, which increases both training time and resource usage in terms of time and space.

Problem 2) The Q-value of each action in a specific state determines which action the agent should take. Parameters such as the number of empty states can be made effective by assigning strategic initial Q-values to the Q-table instead of using random initial values or starting with a zero table. For example, considering symmetry in the table can reduce the number of states for which Q-values need to be calculated. This approach allows the agent to act more efficiently and converge to the true Q-table more quickly.

Problem 3) The map can be represented with the following symbols: 0 for an empty cell, 1 for cells occupied by X, and -1 for cells occupied by O. The hash of the map in each state can then be used as its unique representation.

Problem 4) To optimize the Q table, the following points may be helpful:

- If there is more than one extra "X" compared to "O," or vice versa, it indicates an invalid state. In other words, one should consider the balance between the players and avoid generating all possible combinations of 1, -1, and 0 on the board.
- Equivalent game situations exist due to different symmetries and rotations. These should be mapped into a single state in the table.

Problem 5) Consider a reward of 5 for an action leading to a winning state, -5 for an action leading to a losing state, and -0.2 for an action resulting in a tie. Additionally, assume a penalty of -0.1 for each general action to discourage unnecessary moves.

Problem 6) To model a Tic Tac Toe game as a weighted graph, we can represent each cell as a node and connect adjacent cells with edges. Weights are assigned based on the game state: +1 if both adjacent nodes are occupied by the same player, -1 if occupied by different players, and 0 if one or both nodes are empty (with the option to refine this with distinct values). This graph-based approach simplifies detecting win/loss conditions by analyzing edge weights, potentially allowing for early reward updates in Q-learning by identifying moves that lead to winning outcomes in advance.